Hw 01 - 2P

Summary of polymorphism, interfaces and abstraction

POLYMORPHISM

Polymorphism means "many forms". It allows objects of different subclasses to be treated as objects of a common superclass while performing different behaviors

Characteristics

- Objects can share the same method name but execute different implementations
- It commonly uses inheritance and method overriding
- The method executed depends on the actual object at runtime
- It makes code shorter, cleaner, more flexible, and easier to maintain

Types of methods

- Overriding: A subclass provides a new implementation of an inherited method
- Overloading: Multiple methods have the same name but different parameters

Example: In a chess game, Pawn and Rook inherit from ChessPiece. Both use the method getPossibleMoves(), but each piece moves differently

INTERFACES

An Interface defines a common contract that classes can implement. It specifies functionality that implementing classes must provide

Notes

Escaneado con CamScanner

Characteristics
- Interfaces define common functionality for different classes.
- In Java, a class uses the keyword implements
- An interface can inherit from another interface.
- Interfaces support abstraction and can be used to achieve polymorphism
- An interface cannot be instantiated directly

Advantages: Interfaces improve flexibility, code organization, reusability, and maintainability.

Example: The interface IFarmAnimal can be implemented by a Chicken class. The class provides the behavior required by the interface.

ABSTRACTION

Abstraction is the process of focusing on the essential characteristics of an object while hiding or ignoring unnecessary details.
In software development, abstraction exposes only the information and functionality that other components need.

Main characteristics
- focuses on relevant information
- Hides unnecessary implementation details
- Simplifies complex systems.
- Helps organize classes and components.
- Improves code understanding, testing, maintenance, and reusability.

Common problems
- Leaky abstraction: Other components depend on internal details that should be hidden.
- Over-abstraction: Too many details are omitted, limiting the functionality available to users.

Example: In a football tournament system, a Person class might include name, age and identification. Details unrelated to the tournament are ignored.

Escaneado con CamScanner